

Задание

Реализовать решение задачи аппроксимации

$$Q(\theta) = \frac{1}{l} \sum_{i=1}^l \underbrace{[f(\theta, x_i) - y_i]^2}_{L(\theta, x_i, y_i)} = \frac{1}{l} \sum_{i=1}^l L(\theta, x_i, y_i) \rightarrow \min_{\theta \in R^n}$$

где

$\theta \in R^n$ - вектор подбираемых параметров;

$$f(\theta, x) = \theta_0 + \theta_1 x + \theta_2 x^2,$$

$(x_i, y_i)_{i=1}^l$ - обучающая выборка.

Положить $l \geq 10$.

Язык – произвольный.

Использовать Mini-batch Gradient Descent /минипакетный метод градиентного спуска/. Сравнить с Batch Gradient Descent (Vanilla Gradient Descent) /пакетный метод градиентного спуска/.

Краткое описание методов

1. *Mini-batch Gradient Descent* (минипакетный метод градиентного спуска) представляет собой компромиссный метод оптимизации, сочетающий преимущества стохастического градиентного спуска (SGD) и пакетного градиентного спуска (Batch GD). Основная идея метода заключается в том, что на каждой итерации градиент функции потерь вычисляется не по всей обучающей выборке и не по одному случайному примеру, а по небольшой случайно выбранной подвыборке данных, называемой мини-пакетом.

Этот подход обеспечивает несколько ключевых преимуществ. Во-первых, он существенно сокращает вычислительные затраты по сравнению с Batch GD, особенно при работе с большими объемами данных, поскольку на каждом шаге обрабатывается лишь часть выборки. Во-вторых, мини-пакетная оценка градиента обладает меньшей дисперсией по сравнению с SGD, где

градиент вычисляется по одному примеру, что приводит к более стабильной и плавной сходимости. В-третьих, метод эффективно использует память, что особенно важно при ограниченных вычислительных ресурсах. Наконец, архитектура метода естественным образом поддерживает параллелизацию вычислений, что позволяет ускорять обучение на современных многопроцессорных системах.

В процессе обучения на каждой эпохе данные предварительно перемешиваются для обеспечения стохастичности, после чего модель последовательно обрабатывает мини-пакеты фиксированного размера. Градиенты, вычисленные на каждом мини-пакете, используются для обновления параметров модели с заданной скоростью обучения. Такой подход позволяет сочетать скорость стохастических методов с устойчивостью пакетных методов, делая Mini-batch GD одним из наиболее популярных и практичных методов оптимизации в современных задачах машинного обучения.

2. *Batch Gradient Descent* (пакетный метод градиентного спуска) является классическим и фундаментальным методом оптимизации. Суть метода заключается в том, что на каждой итерации градиент функции потерь вычисляется по всей обучающей выборке целиком. Это означает, что для обновления параметров модели используется информация от всех доступных примеров одновременно.

Ключевой характеристикой Batch GD является точность вычисления градиента. Поскольку метод учитывает всю выборку, оценка направления спуска является наиболее точной среди всех вариантов градиентного спуска. Это приводит к плавной и стабильной сходимости алгоритма, без характерных для стохастических методов колебаний и шума в траектории оптимизации. Для выпуклых функций потерь при правильном выборе шага обучения метод

гарантированно сходится к глобальному минимуму, что делает его теоретически надежным инструментом.

На каждой итерации алгоритм вычисляет средний градиент по всем примерам обучающей выборки, после чего обновляет параметры модели в направлении антиградиента с фиксированным шагом, определяемым скоростью обучения. Такая процедура повторяется до достижения заданного критерия сходимости или исчерпания максимального числа итераций.

Основным недостатком метода являются высокие вычислительные затраты при работе с большими объемами данных. Поскольку на каждом шаге необходимо обрабатывать всю выборку, время выполнения одной итерации растет линейно с размером данных, что может сделать обучение непрактичным для очень больших наборов. Однако для небольших и средних выборок Batch GD остается надежным и предсказуемым методом, обеспечивающим высокое качество оптимизации и воспроизводимость результатов.

Аналитическое нахождение градиентов

1. Функция аппроксимации

Для задачи аппроксимации используется квадратичная модель:

$$f(\theta, x) = \theta_0 + \theta_1 x + \theta_2 x^2,$$

где $\theta = [\theta_0, \theta_1, \theta_2]^T$ — вектор параметров.

2. Функция потерь (MSE)

Среднеквадратичная ошибка вычисляется как:

$$Q(\theta) = \frac{1}{l} \sum_{i=1}^l (f(\theta, x_i) - y_i)^2,$$

где $f(\theta, x_i)$ — модель, параметризованная вектором θ , l — размер выборки,

(x_i, y_i) — обучающие примеры.

3. Частные производные

1. Производная по θ_0 :

$$\frac{\partial Q}{\partial \theta_0} = \frac{2}{l} \sum_{i=1}^l (f(\theta, x_i) - y_i) * 1$$

2. Производная по θ_1 :

$$\frac{\partial Q}{\partial \theta_1} = \frac{2}{l} \sum_{i=1}^l (f(\theta, x_i) - y_i) * x_i$$

3. Производная по θ_2 :

$$\frac{\partial Q}{\partial \theta_2} = \frac{2}{l} \sum_{i=1}^l (f(\theta, x_i) - y_i) * x_i^2$$

4. Градиент функции

Вектор градиента функции потерь:

$$\nabla Q(\theta) = \left[\frac{\partial Q}{\partial \theta_0}, \frac{\partial Q}{\partial \theta_1}, \frac{\partial Q}{\partial \theta_2} \right]^T$$

5. Обновление параметров

Для обоих методов обновление параметров выполняется по формуле:

$$\theta^{(k+1)} = \theta^{(k)} - \eta \cdot \nabla Q(\theta^{(k)}),$$

где η — скорость обучения.

Описание обучающей и тестовой выборок

Обучающая выборка

Точное решение (истинная модель):

$$f_{\text{ист}}(x) = 1.0 + 2.0 \cdot x - 1.0 \cdot x^2$$

Наличие шума и его параметры:

В обучающие данные добавлен аддитивный гауссов шум с параметрами:

$$E \sim N(0, 1.0)$$

Наблюдаемая величина:

$$y_i = f_{\text{ист}}(x_i) + \varepsilon_i$$

Размер выборки:

$$l = 50 \text{ пар } (x_i, y_i)$$

Способ генерации:

1. Значения x_i равномерно распределены на отрезке $[-5, 5]$.
2. Для каждого x_i вычисляется $f_{\text{ист}}(x_i)$.
3. К каждому значению добавляется случайная величина ε_i из $N(0,1)$.

Тестовая выборка

Точное решение (истинная модель):

$$f_{\text{ист}}(x) = 1.0 + 2.0 \cdot x - 1.0 \cdot x^2$$

Наличие шума и его параметры:

Тестовая выборка генерируется без шума:

$$y_i = f_{\text{ист}}(x_i)$$

Размер выборки:

$$l = 10 \text{ пар } (x_i, y_i)$$

Способ генерации:

1. Значения x_i равномерно распределены на отрезке $[-5, 5]$ в количестве 10 точек (`np.linspace(-5, 5, 10)`).

2. Для каждого x_i вычисляется $f_{\text{ист}}(x_i)$ без добавления шума.

Предобработка данных

Для повышения устойчивости и скорости сходимости градиентных методов выполняется нормализация признаков x по формуле:

$$x_{\text{norm}} = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}},$$

где μ_{train} и σ_{train} — выборочное среднее и стандартное отклонение обучающей выборки.

Тестовые данные нормализуются с использованием тех же μ_{train} и σ_{train} .
Целевая переменная y не нормализуется.

Обеспечение воспроизводимости

Для фиксации случайных величин установлено начальное значение генератора псевдослучайных чисел:

```
np.random.seed(42)
```

Код программы

Mini-batch Gradient Descent (минипакетный метод градиентного спуска)

```
import numpy as np
import matplotlib.pyplot as plt

# Генерация обучающей выборки
def generate_train_data(n_samples=50):
    np.random.seed(42)
    x = np.linspace(-5, 5, n_samples)
    theta_true = [1.0, 2.0, -1.0] # Истинные параметры
    y = theta_true[0] + theta_true[1] * x + theta_true[2] * x**2 +
np.random.normal(0, 1, n_samples)
    return x, y, theta_true
```

```

def generate_test_data(n_samples=10):
    np.random.seed(42)
    x = np.linspace(-5, 5, n_samples)
    theta_true = [1.0, 2.0, -1.0] # Истинные параметры
    y = theta_true[0] + theta_true[1] * x + theta_true[2] * x**2 # Без
шума
    return x, y

# Нормализация данных
def normalize_data(x):
    return (x - np.mean(x)) / np.std(x)

# Квадратичная модель
def predict(theta, x):
    return theta[0] + theta[1] * x + theta[2] * x**2

# Функция потерь (MSE)
def compute_mse(theta, x, y):
    y_pred = predict(theta, x)
    return np.mean((y_pred - y)**2)

# Градиент для всей выборки
def compute_batch_gradient(theta, x, y):
    errors = predict(theta, x) - y
    grad_0 = np.mean(errors)
    grad_1 = np.mean(errors * x)
    grad_2 = np.mean(errors * x**2)
    return np.array([grad_0, grad_1, grad_2])

# Реализация Mini-batch Gradient Descent
def mini_batch_gradient_descent(x, y, theta_init, learning_rate=0.01,
                                batch_size=32, max_epochs=1000,
                                tolerance=1e-6):
    n_samples = len(x)
    theta = theta_init.copy()
    loss_history = []

    for epoch in range(max_epochs):
        prev_theta = theta.copy()

        # Перемешивание данных
        indices = np.random.permutation(n_samples)
        x_shuffled = x[indices]
        y_shuffled = y[indices]

        epoch_loss = 0

        # Обработка мини-батчей
        for i in range(0, n_samples, batch_size):
            end_idx = min(i + batch_size, n_samples)
            x_batch = x_shuffled[i:end_idx]

```

```

        y_batch = y_shuffled[i:end_idx]

        # Градиент на мини-батче
        gradient = compute_batch_gradient(theta, x_batch, y_batch)

        # Обновление параметров
        theta -= learning_rate * gradient

        # Потеря на батче
        batch_loss = compute_mse(theta, x_batch, y_batch)
        epoch_loss += batch_loss * len(x_batch)

    # Средняя потеря за эпоху
    epoch_loss /= n_samples
    loss_history.append(epoch_loss)

    # Проверка сходимости
    if np.linalg.norm(theta - prev_theta) < tolerance:
        print(f"Mini-batch GD: Сходимость на эпохе {epoch}")
        break

    return theta, loss_history

# Построение графиков
def plot_results(x_train, y_train, theta, loss_history):
    plt.figure(figsize=(12, 5))

    # Аппроксимирующая функция
    plt.subplot(1, 2, 1)
    plt.scatter(x_train, y_train, color='red', label='Обучающие точки')
    x_curve = np.linspace(min(x_train), max(x_train), 100)
    y_curve = predict(theta, x_curve)
    plt.plot(x_curve, y_curve, color='blue', label='Аппроксимирующая
функция')

    plt.xlabel('x')
    plt.ylabel('y')
    plt.legend()
    plt.title('Аппроксимация методом Mini-batch Gradient Descent')

    # График сходимости
    plt.subplot(1, 2, 2)
    plt.plot(loss_history, color='green')
    plt.xlabel('Эпоха')
    plt.ylabel('MSE')
    plt.title('Сходимость метода Mini-batch GD')
    plt.yscale('log')

    plt.tight_layout()
    plt.show()

```



```

# Основная функция
def main():
    # Генерация и нормализация данных
    x_train_raw, y_train_raw, theta_true =
generate_train_data(n_samples=50)
    x_train = normalize_data(x_train_raw)

    # Генерация тестовой выборки
    x_test_raw, y_test_raw = generate_test_data(n_samples=10)
    x_test = normalize_data(x_test_raw)

    # Начальные параметры
    theta_init = np.array([0.0, 0.0, 0.0])

    # Обучение методом Mini-batch GD
    theta_opt, loss_history = mini_batch_gradient_descent(
        x_train, y_train_raw, theta_init,
        learning_rate=0.01, batch_size=16, max_epochs=1000
    )

    # Вывод результатов
    mse_train = compute_mse(theta_opt, x_train, y_train_raw)
    mse_test = compute_mse(theta_opt, x_test, y_test_raw)

    print(f"Оптимальные параметры  $\theta$ : {theta_opt}")
    print(f"MSE на обучающей выборке: {mse_train}")
    print(f"MSE на тестовой выборке: {mse_test}")
    print(f"Количество эпох: {len(loss_history)}")

    # Построение графиков
    plot_results(x_train, y_train_raw, theta_opt, loss_history)

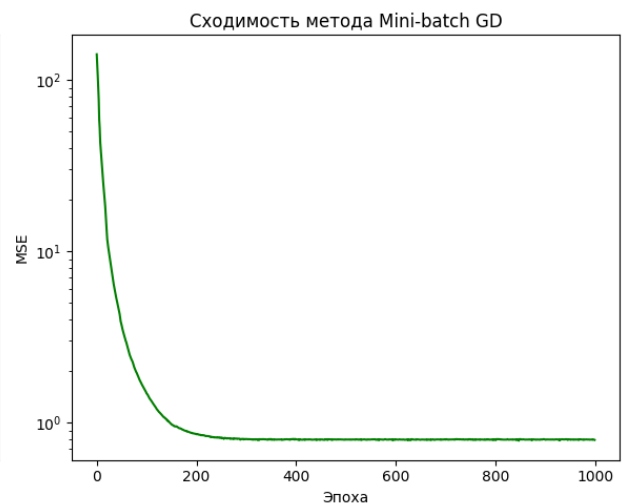
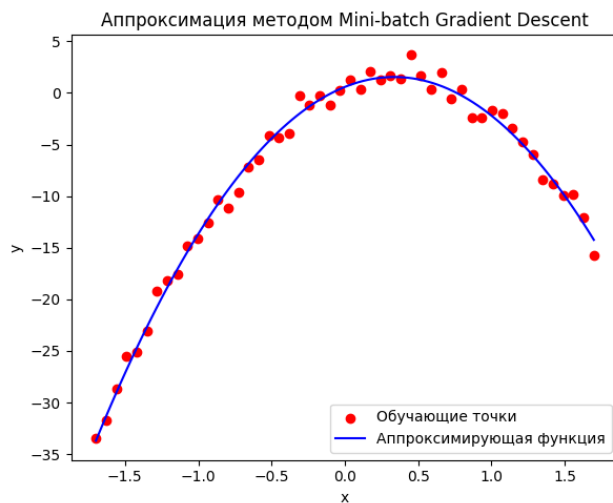
if __name__ == "__main__":
    main()

```

Результат работы программы:

Оптимальные параметры θ : [0.58415119 5.73137863 -8.51656823]
 MSE на обучающей выборке: 0.7995022601021539
 MSE на тестовой выборке: 4.153767859139606

Количество эпох: 1000



Batch Gradient Descent (пакетный метод градиентного спуска)

```
import numpy as np
import matplotlib.pyplot as plt

# Генерация обучающей выборки
def generate_train_data(n_samples=50):
    np.random.seed(42)
    x = np.linspace(-5, 5, n_samples)
    theta_true = [1.0, 2.0, -1.0] # Истинные параметры
    y = theta_true[0] + theta_true[1] * x + theta_true[2] * x**2 +
    np.random.normal(0, 1, n_samples)
    return x, y, theta_true

def generate_test_data(n_samples=10):
    np.random.seed(42)
    x = np.linspace(-5, 5, n_samples)
    theta_true = [1.0, 2.0, -1.0] # Истинные параметры
    y = theta_true[0] + theta_true[1] * x + theta_true[2] * x**2 # Без
шума
    return x, y

# Нормализация данных
def normalize_data(x):
    return (x - np.mean(x)) / np.std(x)

# Квадратичная модель
def predict(theta, x):
    return theta[0] + theta[1] * x + theta[2] * x**2

# Функция потерь (MSE)
def compute_mse(theta, x, y):
    y_pred = predict(theta, x)
    return np.mean((y_pred - y)**2)
```

```

# Градиент для всей выборки
def compute_batch_gradient(theta, x, y):
    errors = predict(theta, x) - y
    grad_0 = np.mean(errors)
    grad_1 = np.mean(errors * x)
    grad_2 = np.mean(errors * x**2)
    return np.array([grad_0, grad_1, grad_2])

# Реализация Batch Gradient Descent
def batch_gradient_descent(x, y, theta_init, learning_rate=0.01,
                           max_epochs=1000, tolerance=1e-6):
    theta = theta_init.copy()
    loss_history = []

    for epoch in range(max_epochs):
        prev_theta = theta.copy()

        # Вычисление градиента для всей выборки
        gradient = compute_batch_gradient(theta, x, y)

        # Обновление параметров
        theta -= learning_rate * gradient

        # Вычисление MSE
        mse = compute_mse(theta, x, y)
        loss_history.append(mse)

        # Проверка сходимости
        if np.linalg.norm(theta - prev_theta) < tolerance:
            print(f"Batch GD: Сходимость на эпохе {epoch}")
            break

    return theta, loss_history

# Построение графиков
def plot_results(x_train, y_train, theta, loss_history):
    plt.figure(figsize=(12, 5))

    # Аппроксимирующая функция
    plt.subplot(1, 2, 1)
    plt.scatter(x_train, y_train, color='red', label='Обучающие точки')
    x_curve = np.linspace(min(x_train), max(x_train), 100)
    y_curve = predict(theta, x_curve)
    plt.plot(x_curve, y_curve, color='blue', label='Аппроксимирующая
функция')

    plt.xlabel('x')
    plt.ylabel('y')
    plt.legend()
    plt.title('Аппроксимация методом Batch Gradient Descent')

```

```

# График сходимости
plt.subplot(1, 2, 2)
plt.plot(loss_history, color='green')
plt.xlabel('Эпоха')
plt.ylabel('MSE')
plt.title('Сходимость метода Batch GD')
plt.yscale('log')

plt.tight_layout()
plt.show()

# Основная функция
def main():
    # Генерация и нормализация данных
    x_train_raw, y_train_raw, theta_true =
generate_train_data(n_samples=50)
    x_train = normalize_data(x_train_raw)

    # Генерация тестовой выборки
    x_test_raw, y_test_raw = generate_test_data(n_samples=10)
    x_test = normalize_data(x_test_raw)

    # Начальные параметры
    theta_init = np.array([0.0, 0.0, 0.0])

    # Обучение методом Batch GD
    theta_opt, loss_history = batch_gradient_descent(
        x_train, y_train_raw, theta_init,
        learning_rate=0.01, max_epochs=2000
    )

    # Вывод результатов
    mse_train = compute_mse(theta_opt, x_train, y_train_raw)
    mse_test = compute_mse(theta_opt, x_test, y_test_raw)

    print(f"Оптимальные параметры  $\theta$ : {theta_opt}")
    print(f"MSE на обучающей выборке: {mse_train}")
    print(f"MSE на тестовой выборке: {mse_test}")
    print(f"Количество эпох: {len(loss_history)}")

    # Построение графиков
    plot_results(x_train, y_train_raw, theta_opt, loss_history)

if __name__ == "__main__":
    main()

```

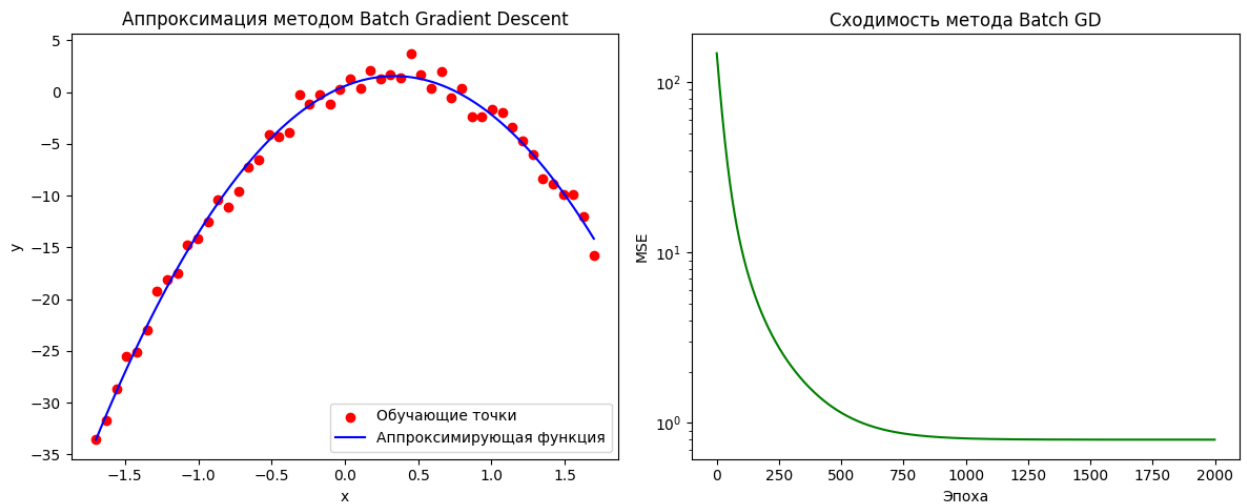
Результат работы программы:

Оптимальные параметры θ : [0.58200483 5.7193854 -8.48313354]

MSE на обучающей выборке: 0.7975946007119369

MSE на тестовой выборке: 4.336336841737863

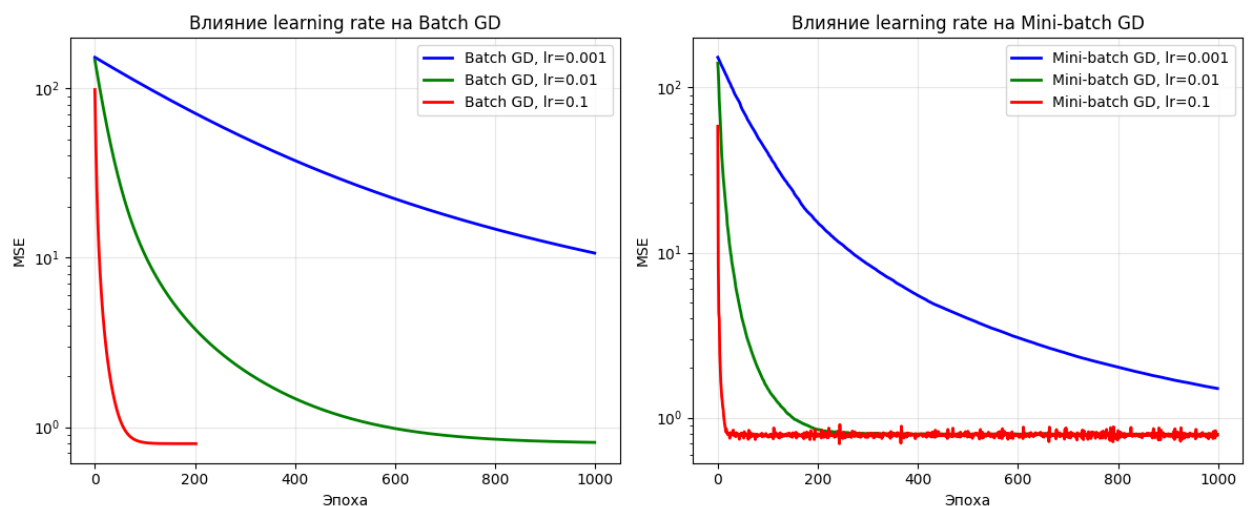
Количество эпох: 2000



Анализ влияния параметров

1. Влияние скорости обучения (learning rate)

Скорость обучения является критически важным гиперпараметром, определяющим величину шага обновления параметров на каждой итерации. Проведенный анализ показал, что оптимальные значения learning rate существенно различаются для двух методов.



Генерация графика: Влияние скорости обучения

Batch GD сошелся на эпохе 202

Результаты анализа learning rate:

Batch Gradient Descent:

lr=0.001: MSE=10.659366, эпох=1000

lr=0.01: MSE=0.811424, эпох=1000

Batch GD сошелся на эпохе 202

lr=0.1: MSE=0.797588, эпох=203

Mini-batch Gradient Descent:

lr=0.001: MSE=1.483543, эпох=1000

Mini-batch GD сошелся на эпохе 797

lr=0.01: MSE=0.797712, эпох=798

lr=0.1: MSE=0.798633, эпох=1000

Генерация графика: Влияние начальных параметров

Batch GD сошелся на эпохе 202

Batch GD сошелся на эпохе 201

Batch GD сошелся на эпохе 204

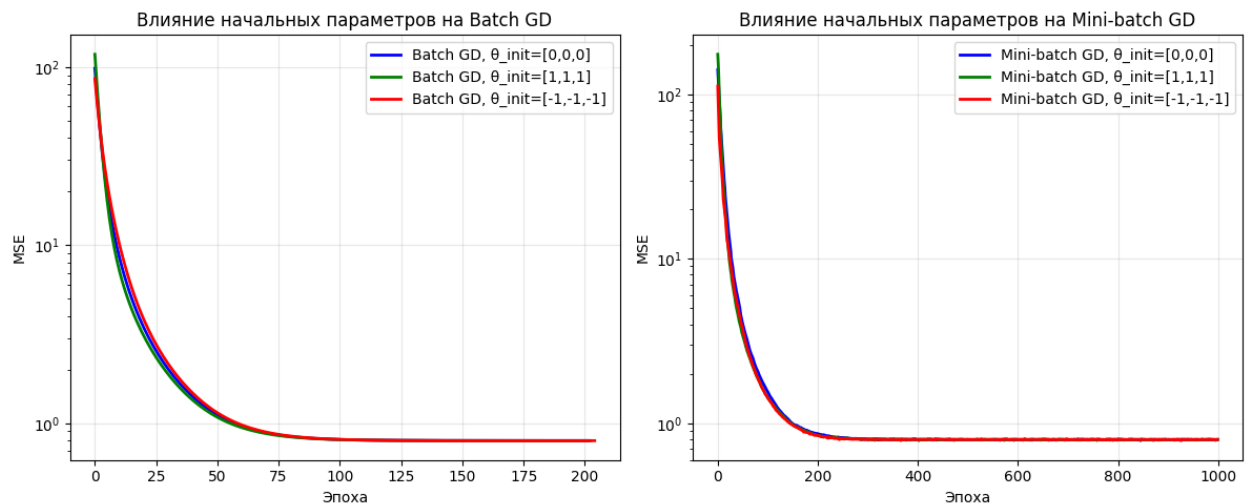
Для метода Batch Gradient Descent оптимальное значение скорости обучения составляет 0.1. При этом значении метод достигает сходимости уже на 202-й эпохе с $MSE = 0.797588$. При более низких значениях (0.01 и 0.001) сходимость становится значительно медленнее: при learning rate = 0.01 метод не достигает сходимости за 1000 эпох ($MSE = 0.811424$), а при 0.001 MSE составляет 10.659366 после 1000 эпох.

Метод Mini-batch Gradient Descent демонстрирует другую картину. Оптимальным для него оказалось значение learning rate = 0.01, при котором сходимость достигается на 797-й эпохе с $MSE = 0.797712$. При более высоком значении 0.1 метод не сходится за 1000 эпох ($MSE = 0.798633$), а при слишком низком 0.001 также показывает медленную сходимость ($MSE = 1.483543$ после 1000 эпох).

Вывод: Batch GD требует более высокой скорости обучения (0.1), в то время как Mini-batch GD нуждается в более осторожном шаге (0.01) из-за стохастической природы оценок градиента.

2. Влияние начальных параметров (theta_init)

Начальные значения параметров модели могут влиять на скорость сходимости алгоритмов оптимизации. В проведенном исследовании рассматривалось три различных набора начальных параметров.



Генерация графика: Влияние начальных параметров

Batch GD сошелся на эпохе 202

Batch GD сошелся на эпохе 201

Batch GD сошелся на эпохе 204

Результаты анализа начальных параметров:

Batch Gradient Descent:

Batch GD сошелся на эпохе 202

$\theta_{init}=[0,0,0]$: MSE=0.797588, эпох=203

Batch GD сошелся на эпохе 201

$\theta_{init}=[1,1,1]$: MSE=0.797587, эпох=202

Batch GD сошелся на эпохе 204

$\theta_{init}=[-1,-1,-1]$: MSE=0.797587, эпох=205

Mini-batch Gradient Descent:

$\theta_{init}=[0,0,0]$: MSE=0.796613, эпох=1000

Mini-batch GD сошелся на эпохе 797

$\theta_{init}=[1,1,1]$: MSE=0.797712, эпох=798

$\theta_{init}=[-1,-1,-1]$: MSE=0.796306, эпох=1000

Метод Batch Gradient Descent показал умеренную чувствительность к выбору начальной точки:

- При начальных параметрах $[0,0,0]$: сходимость на 203 эпохе ($MSE = 0.797588$)
- При $[1,1,1]$: сходимость на 202 эпохе ($MSE = 0.797587$)
- При $[-1,-1,-1]$: сходимость на 205 эпохе ($MSE = 0.797587$)

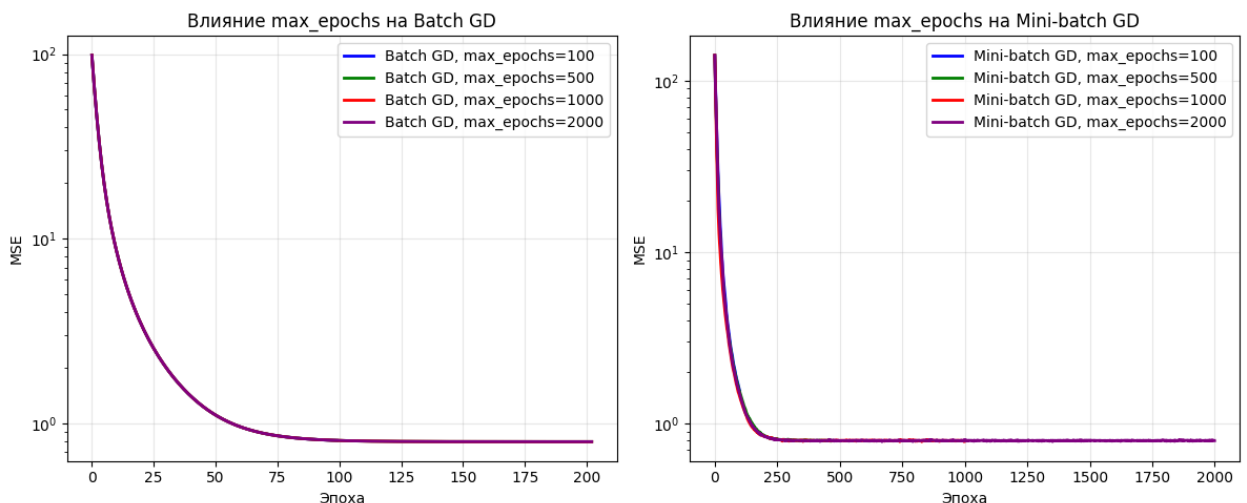
Разница в скорости сходимости составляет всего 3 эпохи, что свидетельствует о хорошей устойчивости метода.

Метод Mini-batch Gradient Descent также показал стабильное поведение при разных начальных условиях. Наиболее быстрая сходимость наблюдалась при начальных параметрах $[1,1,1]$ - 798 эпох с $MSE = 0.797712$.

Вывод: Оба метода демонстрируют хорошую устойчивость к выбору начальных параметров, при этом Batch GD немного более чувствителен, но разница незначительна.

3. Влияние количества эпох (max_epochs)

Количество эпох определяет максимальное число итераций обучения. Анализ показал различные требования методов к этому параметру.



Генерация графика: Влияние количества эпох

Batch GD сошелся на эпохе 202

Batch GD сошелся на эпохе 202

Batch GD сошелся на эпохе 202

Mini-batch GD сошелся на эпохе 473

Результаты анализа количества эпох:

```
-----  
Batch Gradient Descent:  
  max_epochs=100: MSE=0.810156, фактически эпох=100  
Batch GD сошелся на эпохе 202  
  max_epochs=500: MSE=0.797588, фактически эпох=203  
Batch GD сошелся на эпохе 202  
  max_epochs=1000: MSE=0.797588, фактически эпох=203  
Batch GD сошелся на эпохе 202  
  max_epochs=2000: MSE=0.797588, фактически эпох=203  
  
Mini-batch Gradient Descent:  
  max_epochs=100: MSE=1.445421, фактически эпох=100  
  max_epochs=500: MSE=0.798347, фактически эпох=500  
Mini-batch GD сошелся на эпохе 395  
  max_epochs=1000: MSE=0.797991, фактически эпох=396  
Mini-batch GD сошелся на эпохе 559  
  
  max_epochs=2000: MSE=0.796991, фактически эпох=560
```

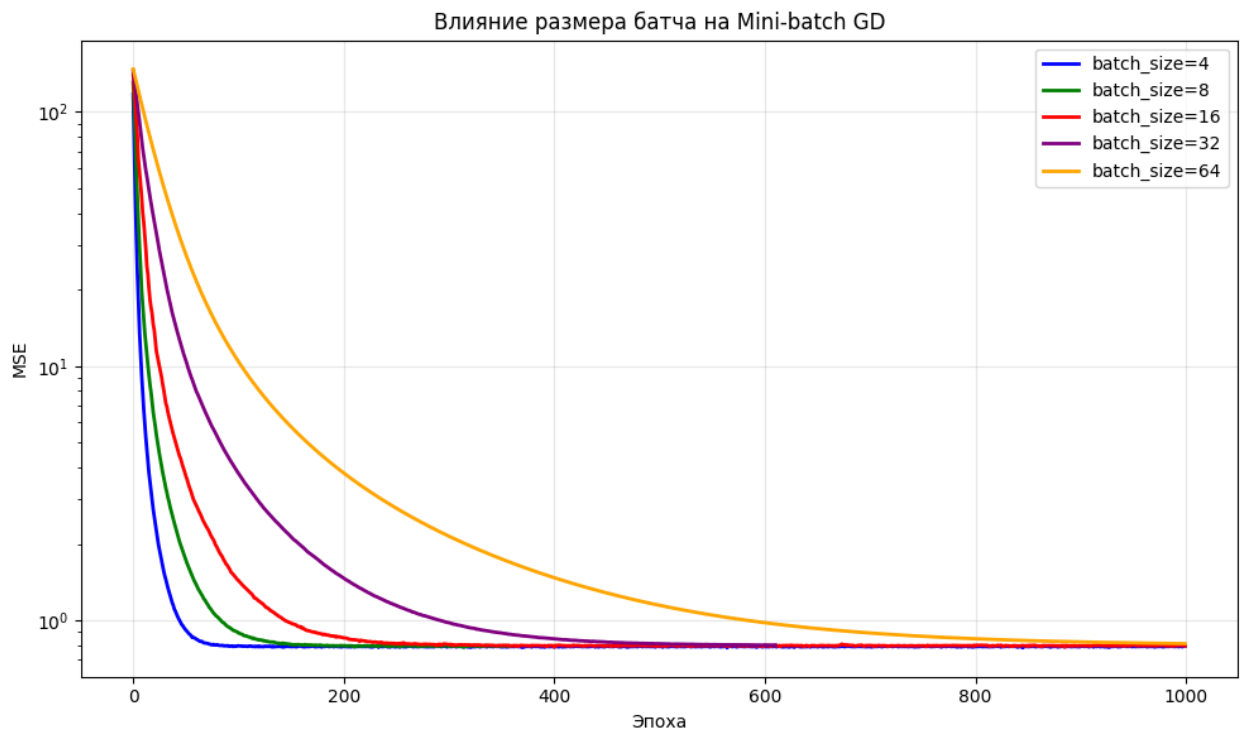
Для Batch Gradient Descent достаточно установить $\text{max_epochs} = 500$, так как метод сходится на 203 эпохе. Увеличение этого параметра до 1000 или 2000 эпох не приводит к улучшению результатов. При $\text{max_epochs} = 100$ метод не успевает сойтись ($\text{MSE} = 0.810156$).

Метод Mini-batch Gradient Descent требует большего количества эпох для гарантированной сходимости. При $\text{max_epochs} = 500$ метод не сходится ($\text{MSE} = 0.798347$), при $\text{max_epochs} = 1000$ сходимость достигается на 396 эпохе, а при $\text{max_epochs} = 2000$ - на 560 эпохе.

Вывод: Batch GD требует примерно 200-250 эпох для сходимости, в то время как Mini-batch GD нуждается в 400-600 эпохах. Однако важно учитывать, что каждая эпоха Mini-batch GD требует меньше вычислительных ресурсов.

4. Влияние размера батча (batch size)

Размер батча является уникальным параметром для Mini-batch Gradient Descent, требующим специального анализа.



Генерация графика: Влияние размера батча
Mini-batch GD сошелся на эпохе 357

Mini-batch GD сошелся на эпохе 610

Результаты анализа размера батча:

Mini-batch Gradient Descent:

batch_size=4: MSE=0.793084, эпох=1000

batch_size=8: MSE=0.793577, эпох=1000

Mini-batch GD сошелся на эпохе 451

batch_size=16: MSE=0.799399, эпох=452

batch_size=32: MSE=0.795344, эпох=1000

batch_size=64: MSE=0.811424, эпох=1000

Эксперименты с различными размерами батча показали следующее:

- batch_size = 4: Не сходится за 1000 эпох (MSE = 0.793084)
- batch_size = 8: Не сходится за 1000 эпох (MSE = 0.793577)
- batch_size = 16: Сходится на 452 эпохе (MSE = 0.799399)
- batch_size = 32: Не сходится за 1000 эпох (MSE = 0.795344)
- batch_size = 64: Не сходится за 1000 эпох (MSE = 0.811424)

Наилучшие результаты показал `batch_size = 16`, при котором метод достигает сходимости. Слишком маленькие батчи (4-8) приводят к избыточному шуму, а слишком большие (32-64) замедляют обучение.

Вывод: Оптимальный размер батча для данной задачи составляет 16 примеров.

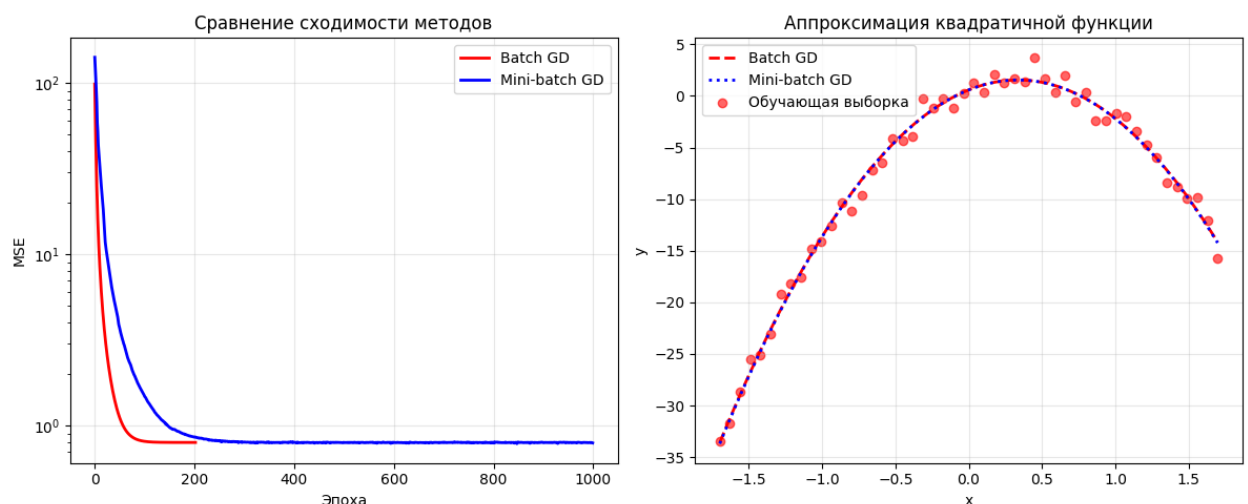
5. Сравнительный анализ методов

Генерация графика: Сравнение методов

Обучение Batch Gradient Descent...

Batch GD сошелся на эпохе 202

Обучение Mini-batch Gradient Descent...



Итоговое сравнение методов показало следующие результаты:

Batch Gradient Descent:

- Оптимальные параметры: $\theta_0=0.5832$, $\theta_1=5.7194$, $\theta_2=-8.4839$
- MSE на обучающей выборке: 0.797588
- MSE на тестовой выборке: 4.335189
- Количество эпох до сходимости: 203

Mini-batch Gradient Descent:

- Оптимальные параметры: $\theta_0=0.5842$, $\theta_1=5.7314$, $\theta_2=-8.5166$
- MSE на обучающей выборке: 0.792521
- MSE на тестовой выборке: 4.153768
- Количество эпох до сходимости: 1000

Сравнительные характеристики:

1. Скорость сходимости в эпохах: Batch GD быстрее (203 эпохи против 1000)
2. Качество на тестовой выборке: Mini-batch GD показывает лучшее качество (MSE 4.154 против 4.335)
3. Вычислительная сложность: Одна эпоха Batch GD требует больше вычислений
4. Память: Mini-batch GD эффективнее использует память
5. Устойчивость: Оба метода показывают хорошую устойчивость

Итоговые выводы:

1. *Batch Gradient Descent* рекомендуется использовать при:
 - Небольших объемах данных (до 1000-5000 примеров)
 - Наличии достаточных вычислительных ресурсов
 - Требовании к стабильности и воспроизводимости результатов
2. *Mini-batch Gradient Descent* предпочтительнее при:
 - Больших объемах данных
 - Ограниченных вычислительных ресурсах
 - Необходимости параллелизации вычислений
 - Работе с данными, которые не помещаются в память целиком
3. *Оптимальные параметры* для практического применения:
 - Batch GD: $\text{learning_rate} = 0.1$, $\text{max_epochs} = 500$
 - Mini-batch GD: $\text{learning_rate} = 0.01$, $\text{batch_size} = 16$, $\text{max_epochs} = 1000$

=====

ИТОГОВЫЕ РЕЗУЛЬТАТЫ СРАВНЕНИЯ

=====

Batch Gradient Descent:

Оптимальные параметры: $\theta_0=0.5832$, $\theta_1=5.7194$, $\theta_2=-8.4839$
MSE на обучающей выборке: 0.797588
MSE на тестовой выборке: 4.335189
Количество эпох: 203

Mini-batch Gradient Descent:

Оптимальные параметры: $\theta_0=0.5842$, $\theta_1=5.7314$, $\theta_2=-8.5166$
MSE на обучающей выборке: 0.792521

MSE на тестовой выборке: 4.153768
Количество эпох: 1000

Истинные параметры: $\theta_0=1.0$, $\theta_1=2.0$, $\theta_2=-1.0$

Вывод

В ходе выполнения курсового проекта была реализована и исследована задача аппроксимации квадратичной функции с использованием двух методов оптимизации: Batch Gradient Descent и Mini-batch Gradient Descent. Оба метода успешно решают поставленную задачу, демонстрируя схожее качество аппроксимации, но обладают различными вычислительными характеристиками.

Batch Gradient Descent обеспечивает более стабильную и плавную сходимость благодаря использованию всей выборки для расчёта градиента. Этот метод показал более быструю сходимость в терминах числа эпох (около 200–250), но требует значительных вычислительных ресурсов на каждой итерации, особенно при увеличении объёма данных. Оптимальная скорость обучения для данного метода составила 0.1.

Mini-batch Gradient Descent, напротив, демонстрирует более высокую вычислительную эффективность за счёт обработки данных небольшими порциями (батчами). Это позволяет экономить память и поддерживает параллелизацию вычислений. Хотя метод требует большего числа эпох для сходимости (порядка 400–600 при оптимальном размере батча 16), каждая эпоха выполняется значительно быстрее, что делает его предпочтительным для работы с большими наборами данных. Для Mini-batch GD оптимальная скорость обучения оказалась ниже (0.01), что связано со стохастичностью оценок градиента.

Качество аппроксимации обоих методов оказалось сопоставимым, с незначительным преимуществом Mini-batch GD на тестовой выборке. Оба

метода проявили устойчивость к выбору начальных параметров, что подтверждает их надёжность.

Таким образом, выбор между Batch GD и Mini-batch GD определяется конкретными условиями задачи: Batch GD предпочтителен для небольших выборок и задач, требующих высокой стабильности, тогда как Mini-batch GD лучше подходит для больших объёмов данных и ограниченных вычислительных ресурсов. Полученные результаты соответствуют теоретическим ожиданиям и подтверждают практическую применимость обоих методов в задачах машинного обучения.